NAMD-LMI Tutorial

Hefei-NAMD with light-matter interaction support

Linjie Chen

2025 June 6 @ Qingdao

University of Science and Technology of China

Light Matter Interaction

When an atom is in the presence of an external *classical electromagnetic fields*, $\mathbf{p} \to \mathbf{p} - e\mathbf{A}$, Hamiltonian becomes:

$$
\begin{aligned}
H &= \frac{(\mathbf{p} - e\mathbf{A})^2}{2m_e} + V_0(\mathbf{r}) \\
&= \left(\frac{p^2}{2m_e} + V_0(\mathbf{r}) \right) + \left(-\frac{e\mathbf{A} \cdot \mathbf{p}}{m_e} + \frac{e^2\mathbf{A}^2}{2m_e} \right) \\
&= H_0 + H_1(t)
\end{aligned}
$$

1. $\mathbf{A}$: vector potential, $\mathbf{E} = -\frac{\partial \mathbf{A}}{\partial t}$;
2. $V_0(r)$: ground-state potential for atom;
3. H_0: ground-state Hamiltonian;
4. H_1: light-matter interaction (LMI) term;
5. $\mathbf{A}^2$ term omittd: $H_1 \approx \frac{e\mathbf{A}\cdot\mathbf{p}}{m_e}$
6. Electric dipole approximation: $\mathbf{A}(\mathbf{r}, t) \approx \mathbf{A}(t)$.

In NAMD, wavefunction of system expanded as

$$|\Psi\rangle = \sum_j |\phi_j\rangle\langle\phi_j|\Psi\rangle = \sum_j c_j|\phi_j\rangle$$

then the system is propagated via

$$i\hbar\frac{\partial|\Psi(\mathbf{r},\mathbf{R},t)\rangle}{\partial t} = H^{\text{tot}}(\mathbf{r},\mathbf{R},t)|\Psi(\mathbf{r},\mathbf{R},t)\rangle$$

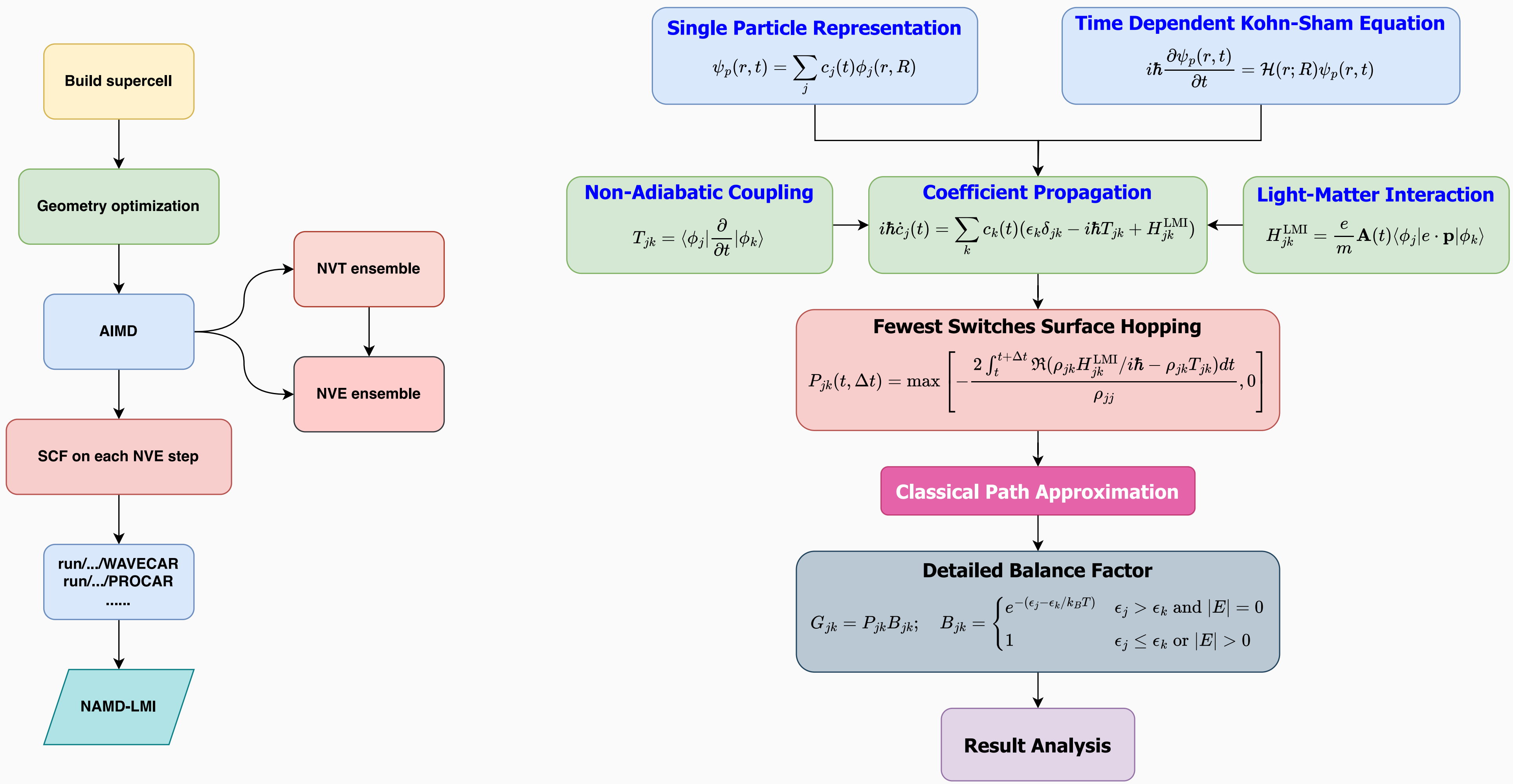
where the total Hamiltonian is given by

$$H^{\text{tot}}(\mathbf{r},\mathbf{R},t) = H^0(\mathbf{r},\mathbf{R},t) + H^{\text{LMI}}(\mathbf{r},\mathbf{R},t)$$

the propagation equation for the expansion foefficients

$$\begin{aligned}\frac{\partial c_{j(t)}}{\partial t} &= \sum_k \left[i\hbar^{-1}\langle\phi_j|H^{\text{tot}}|\phi_k\rangle + \left\langle\phi_j\middle|\frac{\partial}{\partial t}\middle|\phi_k\right\rangle\right]c_{k(t)} \\ &= \sum_k \left[i\hbar^{-1}H^0_{jk}\delta_{jk} + i\hbar^{-1}H^{\text{LMI}}_{jk} + T_{jk}\right]\end{aligned}$$

$H^{\text{LMI}}_{jk} = -\frac{e}{m_e}\langle\phi_j|\mathbf{p}|\phi_k\rangle\cdot\mathbf{A}(t)$ $T_{jk} = \langle\phi_j|\frac{\partial}{\partial t}|\phi_k\rangle$



Workflow of VASP-MD (left) and NAMD-LMI (right)

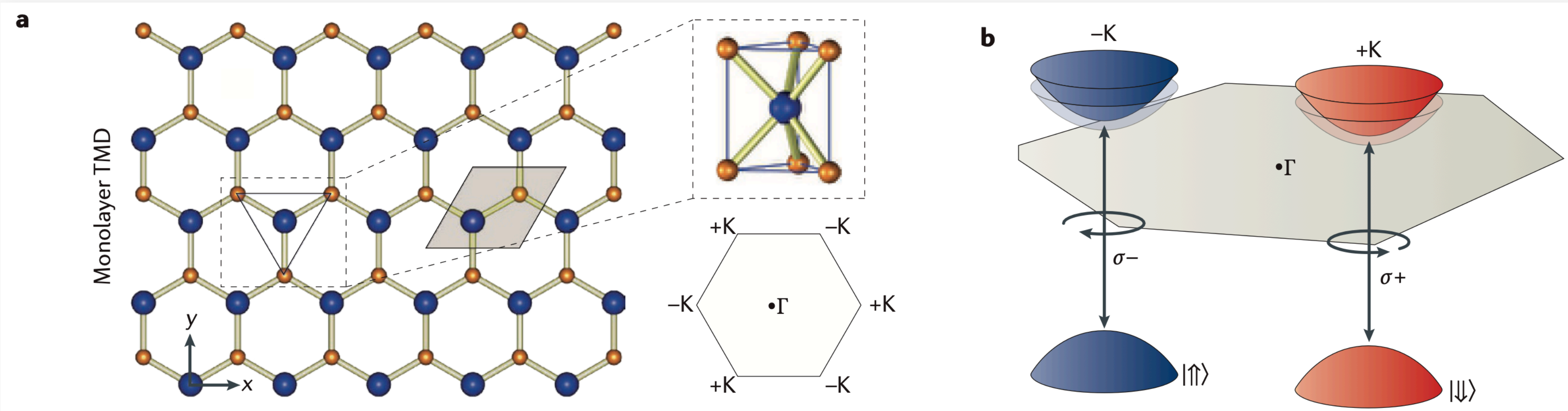
Installation of NAMD-LMI

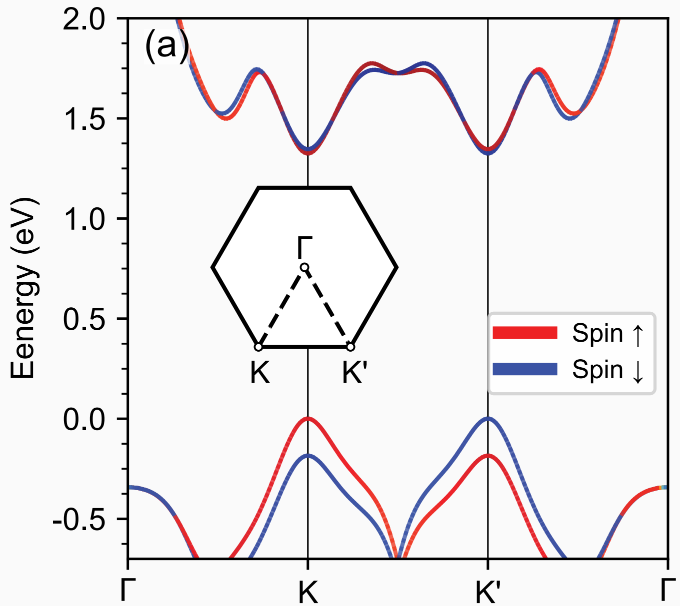
1. Download from GitHub Release (**Recommended**)
   https://github.com/Ionizing/NAMD-LMI/releases
   - `namd_lmi-0.3.1-linux-x86_64-mkl-system.tar.gz` (**Recommended**)
   - `namd_lmi-0.3.1-linux-x86_64-mkl-static.tar.gz`
   - `namd_lmi-0.3.1-macos-x86_64-openblas-static.tar.gz`
   - `namd_lmi-0.3.1-macos-aarch64-openblas-static.tar.gz`
   - `namd_lmi-0.3.1-windows-x86_64-mkl-static.zip`
2. Build from scratch (If necessary)
   - See https://ionizing.github.io/NAMD-LMI/Installation.html

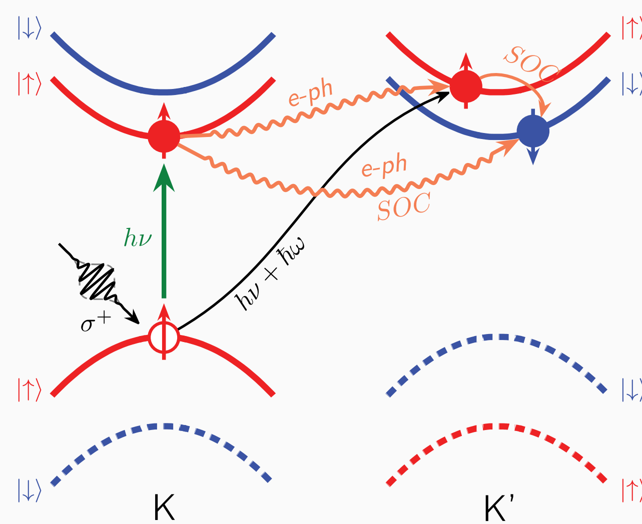
```
$ namd_lmi
+----------------------------------------------------------------------+
|                                                                      |
|    _   _          __  __ _____        _      __  __ _____            |
|   | \ | |   /\   |  \/  |  __ \      | |    |  \/  |_   _|           |
|   |  \| |  /  \  | \  / | |  | |_____| |    | \  / | | |             |
|   | . ` | / /\ \ | |\/| | |  | |_____| |    | |\/| | | |             |
|   | |\  |/ ____ \| |  | | |__| |     | |____| |  | |_| |_            |
|   |_| \_/_/    \_\_|  |_|_____/      |______|_|  |_|_____|           |
|                                                                      |
+----------------------------------------------------------------------+

Welcome to use namd!
    current version:    0.3.1
    git hash:           d7475d6
    author(s):          Ionizing
    host:               aarch64-apple-darwin
    built time:         2025-04-11 23:04:51 +08:00
```

```
Usage: namd_lmi <COMMAND>

Commands:
  waveslice  Slicing WAVECARs and PROCARs from pre-calculated AIMD
trajectory
  nac        Calculate non-adiabatic coupling (NAC) including `<j| d/dt
|k>` and momentum matrix `<i| p |j>`
  hamil      Generate the Hamiltonian from NAC according to config file
  surfhop    Perform the surface-hopping process with given Hamiltonian
file and config file
  help       Print this message or the help of the given subcommand(s)

Options:
  -h, --help     Print help (see more with '--help')
  -V, --version  Print version
```

Example: Photoexcited Spin Valley Dynamics in $MoSe_2$ Monolayer







Nat. Rev. Mater. 1, 16055 (2016); *Adv. Optical Mater.* 2025, 2403069

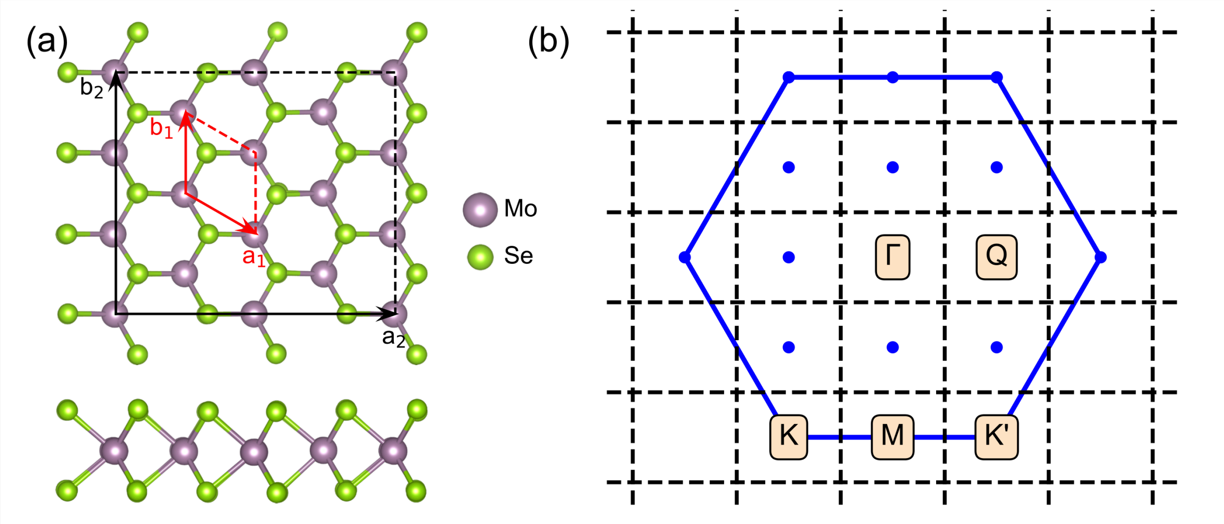
Workflow-VASP: Make supercell

```
#!/usr/bin/env python3
from ase.io import read
from ase.build import make_supercell
M = [[4,2,0],
     [0,3,0],
     [0,0,1]]
primcell = read("POSCAR.prim")
supcell = make_supercell(primcell, P=M, order="atom-major")
supcell.write("POSCAR.sup", vasp5=True, direct=True)
```

```
Mo  S
  1.000
    3.320     0.000     0.000
   -1.660     2.875     0.000
    0.000     0.000    15.000
Mo   Se
1     2
Direct
  0.6564  0.3282  0.5005
  0.9897  0.9948  0.3892
  0.9897  0.9949  0.6117
```

$\Downarrow$

```
Mo Se
  1.000
    9.9600     5.7504     0.0000
   -4.9800     8.6256     0.0000
    0.0000     0.0000    15.0000
Mo   Se
12   24
Direct
  0.1641  0.0000  0.5005
  0.1641  0.3333  0.5005
  0.1641  0.6667  0.5005
  0.4141  0.1667  0.5005
  0.4141  0.5000  0.5005
  ......
```

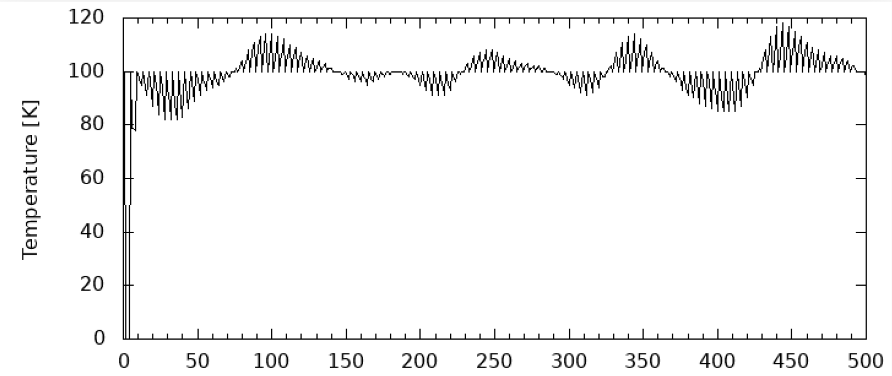


Atomic structure, primitive cell and supercell of $MoSe_2$, and corresponding Brillouin zines.

Workflow-VASP: NVT & NVE

- NVT

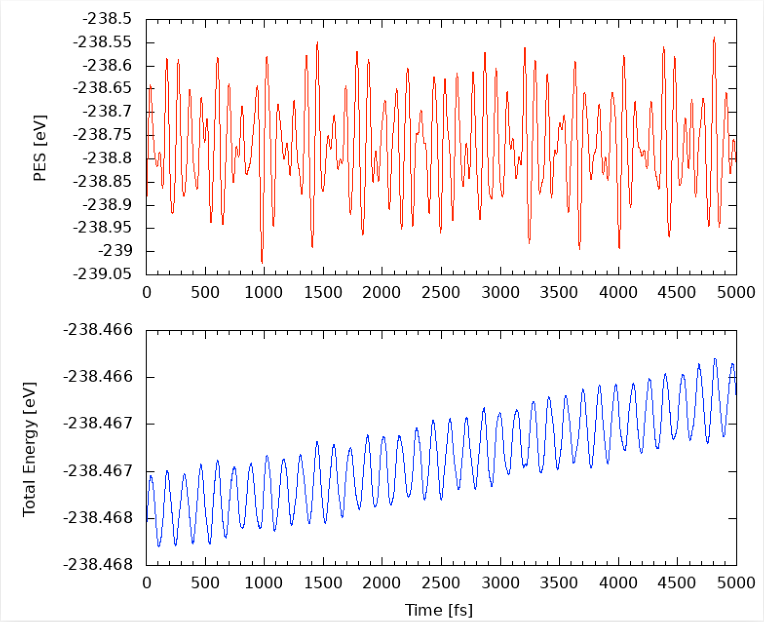
```
...
ISYM   =  0    # turn off symmetry
IBRION =  0    # turn on MD
NSW    =  500  # No. of ionic steps
POTIM  =  1    # time step 1.0 fs
SMASS  = -1    # Canonical
NBLOCK =  4    # Rescale every 4 steps
TEBEG  =  100  # Begin temperature
TEEND  =  100  # End temperature
...
```



Make sure Temperature is stabled at 100 K.

- NVE

```
...
ISYM   =  0     # turn off symmetry
IBRION =  0     # turn on MD
NSW    =  5000  # No. of ionic steps
POTIM  =  1     # time step 1.0 fs
SMASS  = -3     # Microcanonical
NBLOCK =  1     # XDATCAR contains every step
...
```



Total energy is stable. ($\Delta E < 0.01$ eV)

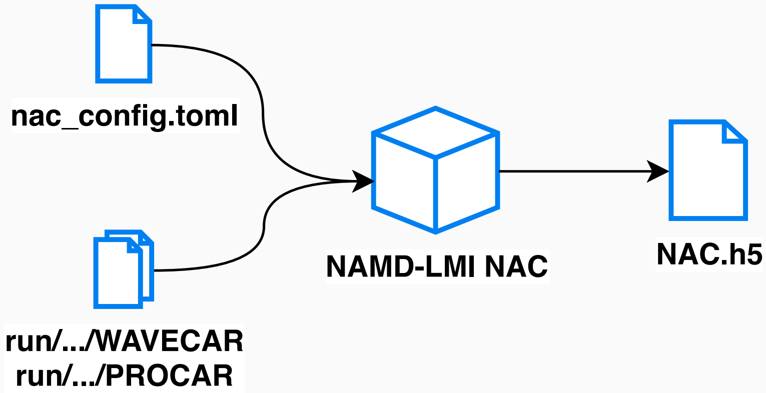
Workflow-VASP: SCF of NVE steps

```
# General
SYSTEM = MoSe2
PREC   = Med
ISPIN  = 1
ISTART = 0
ICHARG = 1

# Electronic relaxation
NPAR   = 8
ISMEAR = 0
SIGMA  = 0.1
ALGO   = Fast
NELMIN = 4
NELM   = 120
EDIFF  = 1E-6
MAGMOM =   0 0 1  0 0 1  0 0 1  0 0 1  0 0 1  0 0 1  0 0 1  0 0 1  0 0 1  0 0 1  0 0 1  0 0 1  72*0

# Molecular Dynamics
ISYM   = 0       # turn off symmetry for MD
IBRION = -1      # turn off MD

# Writing Flag
NWRITE = 2       # make OUTCAR small
LWAVE  = .TRUE.
LCHARG = .TRUE.
LORBIT = 11
LSORBIT = .TRUE.
```

```python
#!/usr/bin/env python3
import os
from ase.io import read, write

print("Reading XDATCAR ...")
CONFIGS = read('XDATCAR', format='vasp-xdatcar', index=':')

NSW    = len(CONFIGS) # The number of ionic steps
NSCF   = 50           # Choose last NSCF steps for SCF calculations
NDIGIT = 4  # len("{:d}".format(NSCF))   #
PREFIX = 'run'        # run directories
DFORM  = "/%%0%dd" % NDIGIT  # run dirctories format
for ii in range(NSCF):       # write POSCARs
    print("Generating {:6d} ...".format(ii+1), end='')
    p = CONFIGS[ii - NSCF]
    r = (PREFIX + DFORM) % (ii + 1)
    if not os.path.isdir(r): os.makedirs(r)
    write('{:s}/POSCAR'.format(r), p, vasp5=True, direct=True)
    print("\b"*21, end='')
```

```
chenlj@SciNat: /data3/chenlj/Work/Work2025/MoSe2_tutorial/2x3/static_ncl
$ ls run/????/POSCAR | wc -l
50
chenlj@SciNat: /data3/chenlj/Work/Work2025/MoSe2_tutorial/2x3/static_ncl
$ \ls -l run/{0001..0050}/WAVECAR | awk '{print $5}' | uniq -c
     50 57714800
chenlj@SciNat: /data3/chenlj/Work/Work2025/MoSe2_tutorial/2x3/static_ncl
$
```

Run SCF on selected NVE trajectories




```
chenlj@SciNat: /data3/chenlj/Work/Work2025/MoSe2_tutorial/2x3/namd  Wed May 28 16:09:53
$ namd_lmi nac --gen conf
2025-05-28 16:09:58 [ INFO] Global logger initialized with targets being stderr and "./globalrun.log"
2025-05-28 16:09:58 [ INFO] Writing `01_nac_config_template.toml ...`
2025-05-28 16:09:58 [ INFO] Time used: 1.423543ms
```

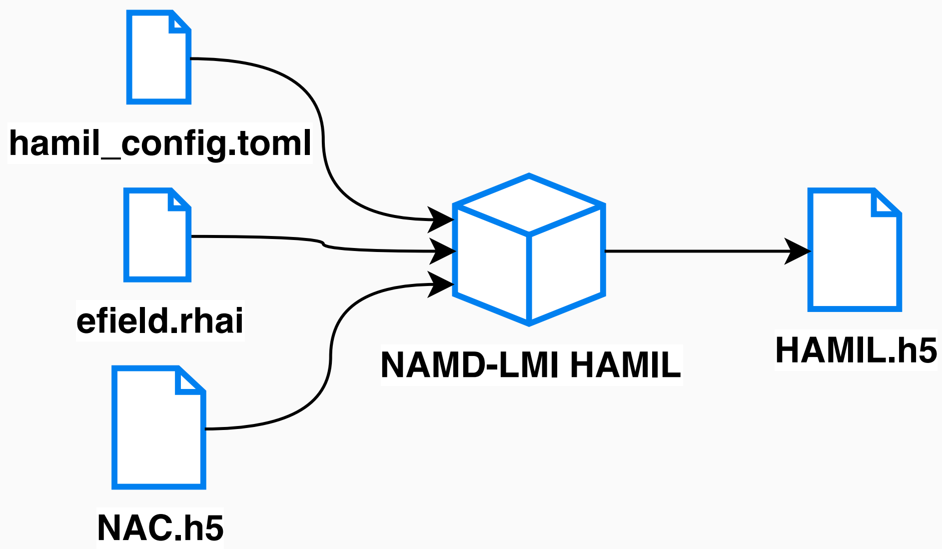
Content of `nac_config.toml`:

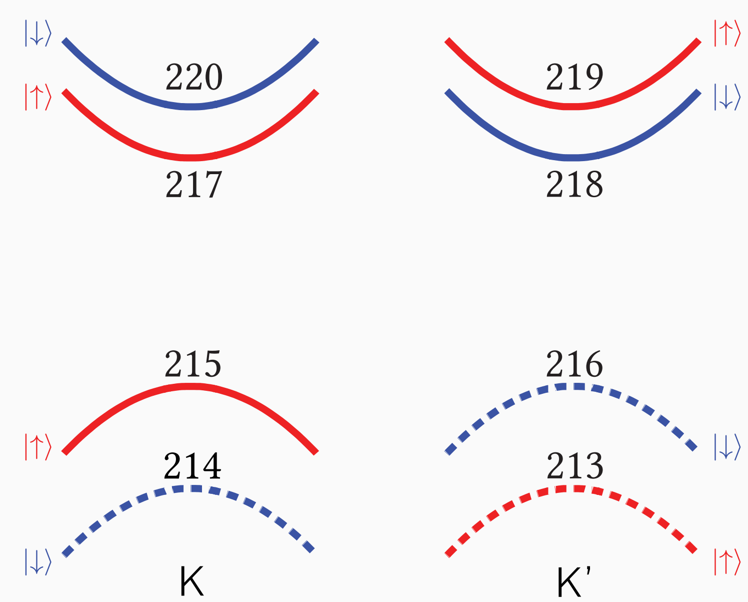
```toml
          rundir = "../static_ncl/run"
         ikpoint = 1
          brange = [213, 220]   # band range
             nsw = 50   # how many SCFs done
          ndigit = 4
           potim = 1    # step length (fs)
     temperature = 100   # Kelvin
   normalization = false
 phasecorrection = true # crucial for MoSe2
        nacfname = "NAC.h5"
```

Run `namd_lmi nac -c nac_config.toml`

```
chenlj@SciNat: /data3/chenlj/Work/Work2025/MoSe2_tutorial/2x3/namd  Wed May 28 16:56:33
$ namd_lmi nac -c 01_nac_config.toml
2025-05-28 16:56:37 [ INFO] Global logger initialized with targets being stderr and "./globalrun.log"
2025-05-28 16:56:37 [ INFO]
+----------------------------------------------------------------------+
|                                                                      |
|   _   _    _    __  __ ____        _     __  __ ___                   |
|  | \ | |  / \  |  \/  |  _ \      | |   |  \/  |_ _|                  |
|  |  \| | / _ \ | |\/| | | | |_____| |   | |\/| || |                   |
|  | |\  |/ ___ \| |  | | |_| |_____| |___| |  | || |                   |
|  |_| \_/_/   \_\_|  |_|____/      |_____|_|  |_|___|                  |
|                                                                      |
+----------------------------------------------------------------------+

Welcome to use namd!
    current version:    0.3.1
    git hash:           d7475d6
    author(s):          Ionizing
    host:               x86_64-unknown-linux-gnu
    built time:         2025-04-11 23:06:26 +08:00

2025-05-28 16:56:37 [ INFO] Running with 0 threads.
2025-05-28 16:56:37 [ INFO] Got NAC config:
####         NAMD-lmi config for Non-Adiabatic Coupling (NAC) calculation       ####
####    YOU NEED TO CHANGE THE PARAMETERS IN THE FOLLOWING TO FIT YOU SYSTEM    ####

              rundir = "../static_ncl/run"
             ikpoint = 1
              brange = [213, 220]
                 nsw = 50
       spin_diabatics = false
              ndigit = 4
               potim = 1
         temperature = 100
       normalization = false
     phasecorrection = true
            nacfname = "NAC.h5"

2025-05-28 16:56:37 [ INFO] No pre-calculated NAC available, start calculating from scratch in "../static_ncl/run"/.../WAVECARs ...
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0001" and "../static_ncl/run/0002" ..., remains:    49
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0015" and "../static_ncl/run/0016" ..., remains:    48
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0028" and "../static_ncl/run/0029" ..., remains:    47
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0010" and "../static_ncl/run/0011" ..., remains:    46
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0029" and "../static_ncl/run/0030" ..., remains:    45
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0020" and "../static_ncl/run/0021" ..., remains:    44
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0026" and "../static_ncl/run/0027" ..., remains:    43
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0016" and "../static_ncl/run/0017" ..., remains:    42
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0007" and "../static_ncl/run/0008" ..., remains:    41
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0013" and "../static_ncl/run/0014" ..., remains:    40
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0030" and "../static_ncl/run/0031" ..., remains:    39
2025-05-28 16:56:37 [ INFO]  Calculating couplings between "../static_ncl/run/0008" and "../static_ncl/run/0009" ..., remains:    38
```

```
In [1]: import h5py

In [2]: f = h5py.File("NAC.h5")

In [3]: f.keys()
Out[3]: <KeysViewHDF5 ['brange', 'efermi', 'eigs', 'ikpoint', 'lncl', 'nbands', 'nbrange', 'ndigit', 'normalization', 'n
spin', 'nsw', 'olaps_i', 'olaps_r', 'phasecorrection', 'pij_i', 'pij_r', 'potim', 'proj', 'spin_diabatics', 'temperature
']>

In [4]: f['efermi'][()]
Out[4]: -0.5031514619481066

In [5]: f['temperature'][()]
Out[5]: 100.0
```





$MoSe_2$ band index.

Content of `hamil_config.toml`:

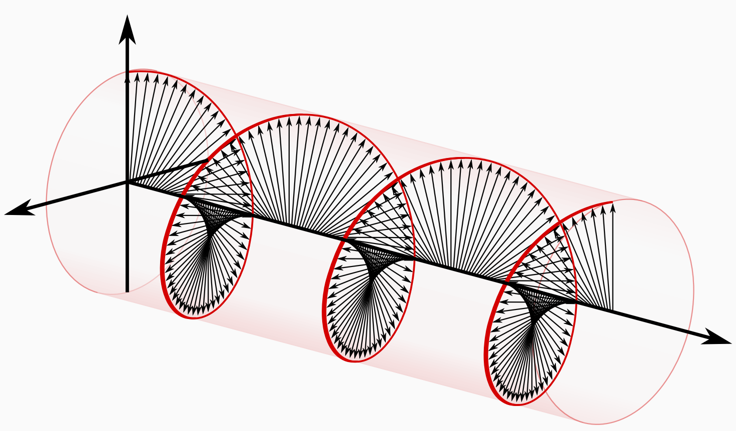
```toml
      ikpoint = 1
   basis_list = "215 217..220"
 basis_labels = ["vKu", "cKu cK'd cK'u cKd"]
spin_diabatics = false
    nac_fname = "NAC.h5"
 efield_fname = "efield.rhai"
  hamil_fname = "HAMIL.h5"
   propmethod = "Expm"
      scissor = 1.71 # unit: eV
```

Photo excitation process is very sensitive to the coupling gap, thus we need to correct it, according to HSE06 gap value (1.71 eV).

Workflow-NAMD-LMI: Hamiltonian (Define optical field)

Under dipole approximation, and using Coulomb gauge, optical field is described by electric field $\mathbf{E} = (E_x, E_y, E_z)^\intercal$.

- For x−polarized light:
  $E_x = E\cos(\omega t); E_y = E_z = 0$
- For σ^+−polarized light:
  $E_x = E\cos(\omega t); E_y = E\sin(\omega t)$
- For σ^-−polarized light:
  $E_x = E\cos(\omega t); E_y = -E\sin(\omega t)$



Left hand circularly (σ^+) polarized light.

Content of `efield.rhai`:

```
fn efield(t) {
  let hbar  = 0.658212;    // reduced planck constant (eV*fs)
  let amp   = 0.007;       // amplitude = 0.005 (Volt/Angstrom)
  let hnu   = 1.71;        // photon energy = h * nu = 1.71 (eV)
  let omega = hnu / hbar;  // omega = hnu / hbar (rad/fs)
  let duration = 1000;     // pulse duration
  let omega_envelop = 2.0 / duration * pi;
  // sigma plus polarized
  let x = amp * cos(omega * t);   // Ex(t)
  let y = amp * sin(omega * t);   // Ey(y)
  let z = 0.0;                    // Ez = 0
  // Hann (sine) pulse
  let envelop = if t <= duration {
    sin(omega_envelop*t - 0.5*pi) * 0.5 + 0.5;
  } else {
    0
  };

  return [
    x*envelop,
    y*envelop,
    z*envelop
  ]; // this statement is required.
}
```

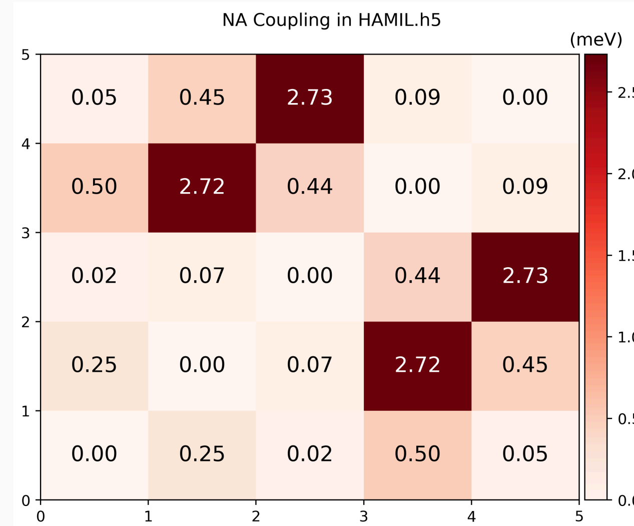
Workflow-NAMD-LMI: Hamiltonian (Run)

```
chenlj@SciNat: /data3/chenlj/Work/Work2025/MoSe2_tutorial/2x3/namd  Thu M
$ namd_lmi hamil -c 02_hamil_config.toml
2025-05-29 15:28:49 [ INFO] Global logger initialized with targets being
2025-05-29 15:28:49 [ INFO]

Welcome to use namd!
    current version:    0.3.1
    git hash:           d7475d6
    author(s):          Ionizing
    host:               x86_64-unknown-linux-gnu
    built time:         2025-04-11 23:06:26 +08:00

2025-05-29 15:28:49 [ INFO] Got Hamiltonnian config:
# NAMD-lmi config for Hamiltonian generation

              ikpoint = 1
           basis_list = [215, 217, 218, 219, 220]
         basis_labels = ["vKu", "cKu cK'd cK'u cKd"]
```

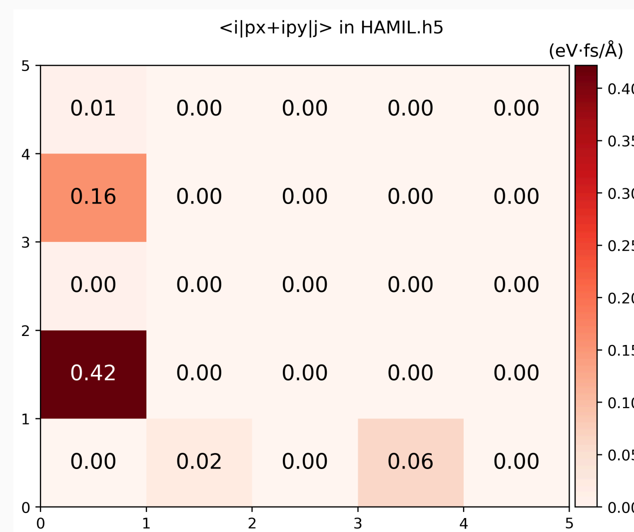
namd_lmi hamil -c hamil_config.toml

```
chenlj@SciNat: /data3/chenlj/Work/Work2025/MoSe2_tutorial/2x3/namd
$ namd_lmi hamil --gen pp
2025-05-29 15:38:27 [ INFO] Global logger initialized with targets
2025-05-29 15:38:27 [ INFO] Writing `hamil_plot.py` ...
2025-05-29 15:38:27 [ INFO] Time used: 1.283828ms
chenlj@SciNat: /data3/chenlj/Work/Work2025/MoSe2_tutorial/2x3/namd
$ ./hamil_plot.py HAMIL.h5
Writing hamil_bands.png
Writing hamil_nac.png
Writing hamil_pij.png
chenlj@SciNat: /data3/chenlj/Work/Work2025/MoSe2_tutorial/2x3/namd
$
```

namd_lmi hamil --gen pp
./hamil_plot.py HAMIL.h5



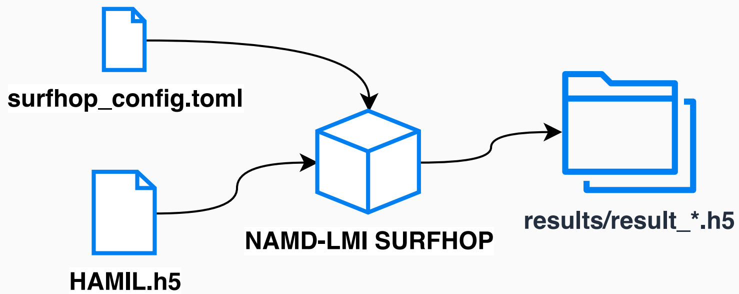
NAC: $\hbar\langle\phi_j|\frac{\partial}{\partial t}|\phi_k\rangle$



TDM: $\langle\phi_j|p_x + ip_y|\phi_k\rangle$ (not Hermitian)

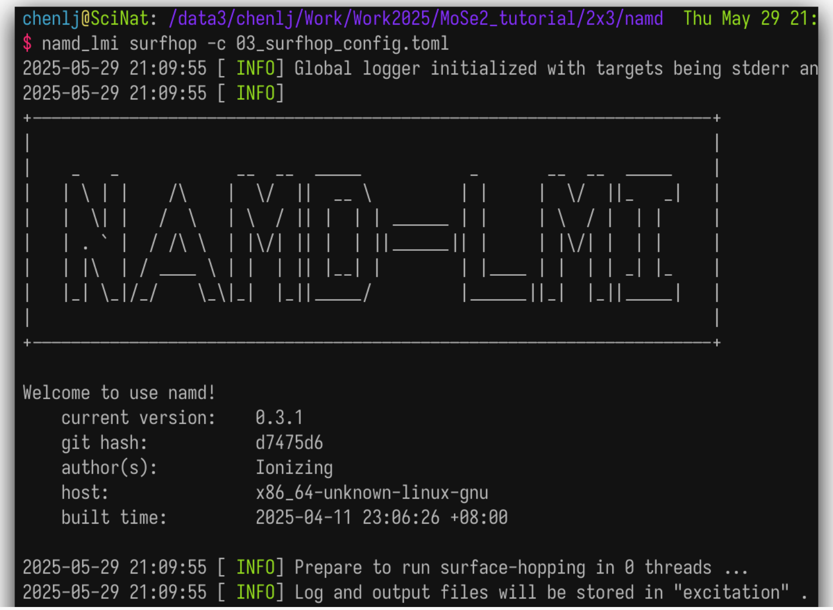
Workflow-NAMD-LMI: Surface Hopping (Run)

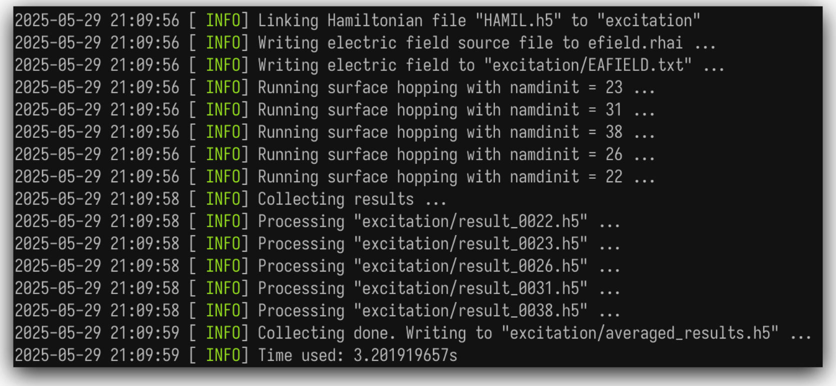
Run `namd_lmi surfhop -c surfhop_config.toml`



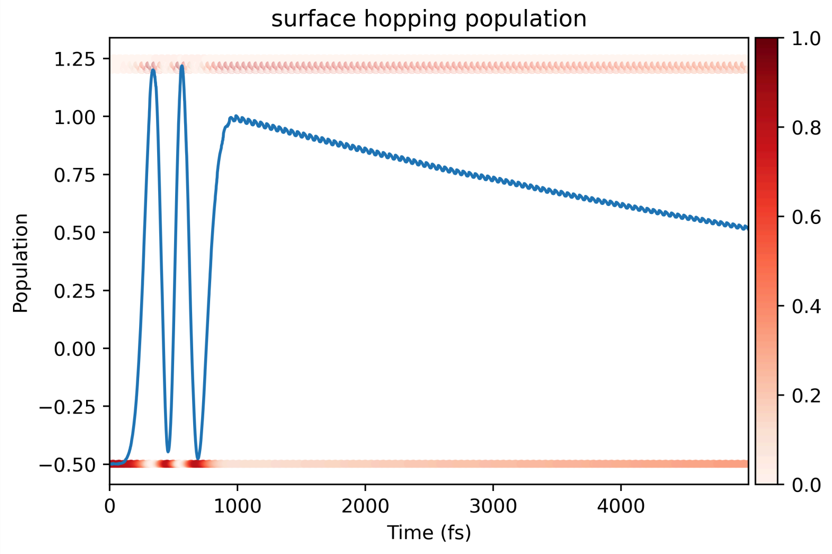
toml

```toml
       hamil_fname = "HAMIL.h5"
          namdtime = 5000
              nelm = 10
             ntraj = 10000
          shmethod = "FSSH"

            outdir = "excitation"
  detailed_balance = "DependsOnEField"
   smearing_method = "LorentzianSmearing"

           iniband = "215"
          inisteps = [
    22 ,
    23 ,
    26 ,
    31 ,
    38
]
```

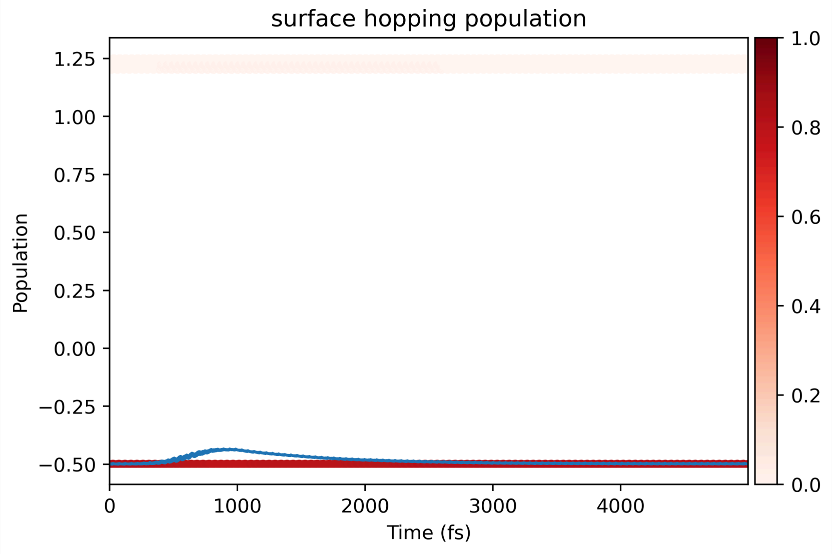




Run `namd_lmi surfhop --gen pp` to generate `surfhop_plot.py`,
then `cd result && ../surfhop_plot.py`, we can get the evolution of electrons.



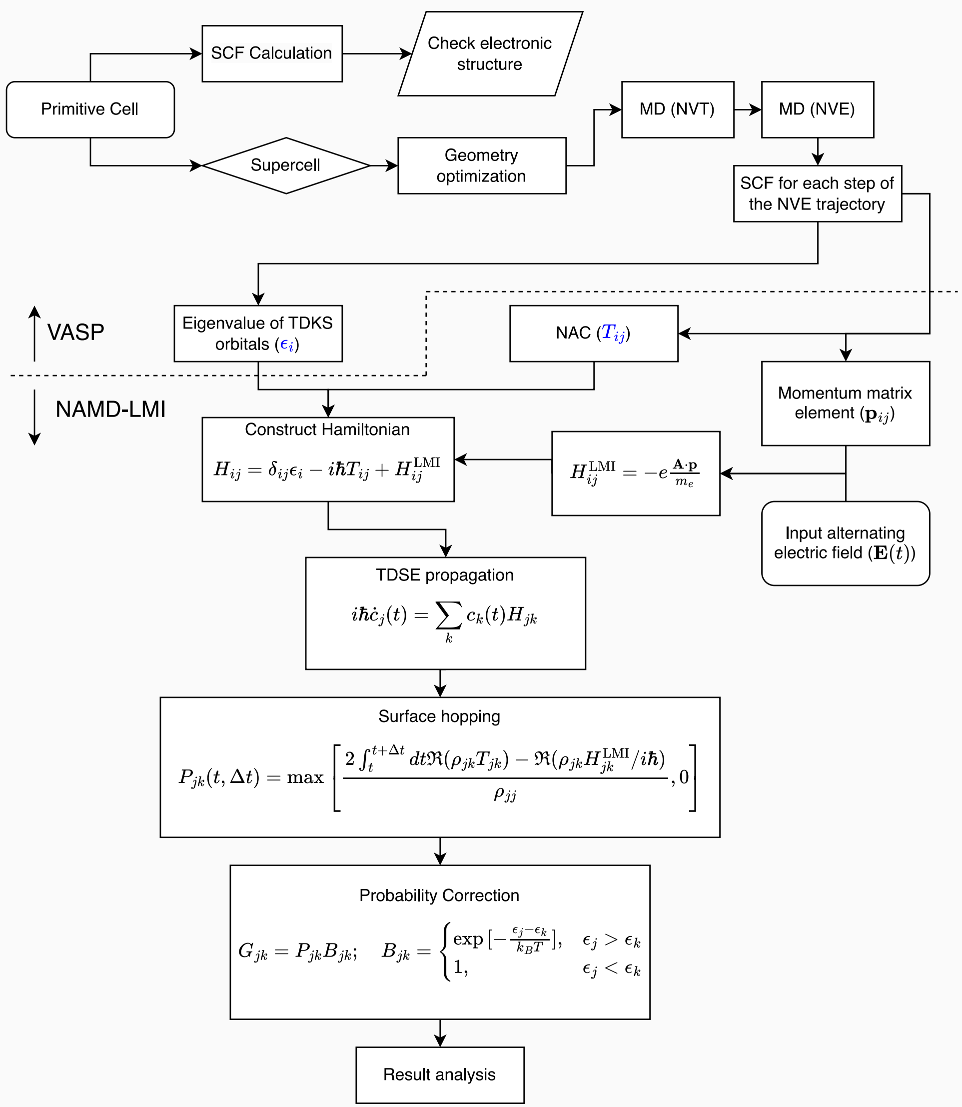
Using σ^+ laser.

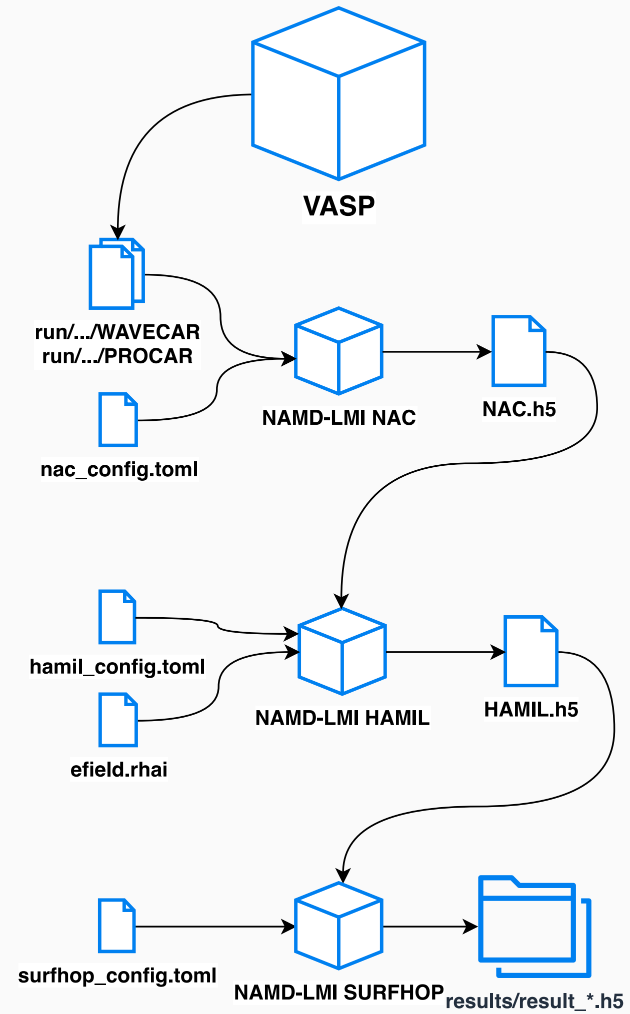


Using σ^- laser.

The blue line denotes the evolution of total energy, where the colors on band denote the time dependent population of each band.

Combined Workflow of NAMD-LMI





Combined workflow of NAMD-LMI.

Overall process of NAMD-LMI.

Remarks

- Almost all input files and post-process files can be generated via `namd_lmi <nac/hamil/surfhop> --gen <missing file>` ;
- Some mysterious thing may be useful: `namd_lmi <nac/hamil/surfhop> --help` ;
- There is a website containing more detailed documentation on NAMD-LMI: https://ionizing.github.io/NAMD-LMI/ ;
- Post process scripts is hardcoded, you may need to modify it to make it run ;
- NAMD-LMI also supports multiple electron dynamics (e.g. `iniband = "213..216"`), and make sure all `iniband` is **IN** your `basis_list` in `hamil_config.toml`.
- NAMD-LMI supports **spin diabatics** representation, just open `spin_diabatics` flag in configs.
- You may need to **regenerate** `HAMIL.h5` when the basis or optical field are changed.
- If you come across any problems, please open an issue at (click new issue) https://github.com/Ionizing/NAMD-LMI/issues ;

Acknowledgement & Readmore

This work cannot be made possible without the help of

- Prof. Jin Zhao (https://staff.ustc.edu.cn/~zhaojin)
- Prof. Qijing Zheng (https://staff.ustc.edu.cn/~zqj)
- Prof. Qunxiang Li (https://staff.ustc.edu.cn/~liqun)
- Dr. Zhi Li (https://hefei-namd.org/author/li-zhi)

NAMD-LMI related

- Paper: https://doi.org/10.1002/adom.202403069
- Source code: https://github.com/Ionizing/NAMD-LMI
- Documentation: https://ionizing.github.io/NAMD-LMI

Other useful resources

- Prof. Jin Zhao's group page:
  https://staff.ustc.edu.cn/~zhaojin / https://hefei-namd.org
- Prof. Qijing Zheng's personal page:
  https://staff.ustc.edu.cn/~zqj / https://github.com/QijingZheng
- Hefei-NAMD source code: https://github.com/QijingZheng/Hefei-NAMD